

Practical No. 7 – Banker's Algorithm

Aim

To write a C program to implement **Banker's Algorithm** to calculate the current need of resources by each process and determine whether the system is in a **safe state** and find the **safe sequence of process execution**.

A computer system has **four types of resources**:

- Printer
- ROM
- Hard Disk
- RAM

The system has **five processes**:

- Google Drive
- Firefox
- Word Processor
- Excel
- PowerPoint

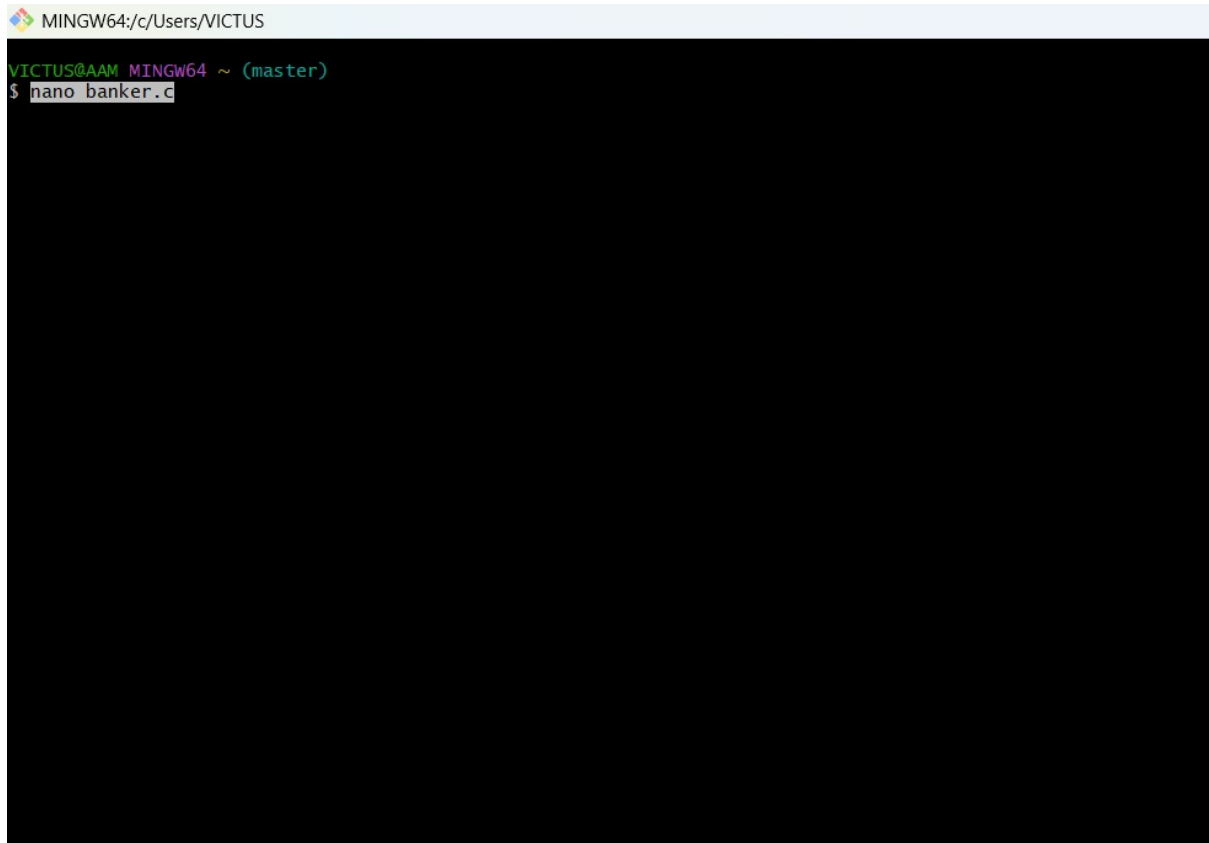
The program should:

1. Calculate the **Need Matrix** (Current need of resources).
2. Check whether the **system is in a safe state** using available resources.
3. If safe, determine the **correct order of process execution (Safe Sequence)**.

Steps Performed

Step 1 – Open Git Bash

First, Git Bash is opened on the system to execute Linux commands and compile the C program.



```
MINGW64:/c/Users/VICTUS
VICTUS@AAM MINGW64 ~ (master)
$ nano banker.c
```

Step 2 – Write the Program

The C program for **Banker's Algorithm** is written in the banker.c file using the Nano editor.

The program includes allocation matrix, maximum matrix, available resources and logic to calculate the **Need Matrix and Safe Sequence**.

```
MINGW64/c/Users/VICTUS/Desktop
GNU nano 8.7 banker.c
#include <stdio.h>

int main() {
    int i,j,k;

    int process = 5;
    int resource = 4;

    int allocation[5][4] = {
        {0,0,1,4},
        {0,6,3,2},
        {0,0,2,2},
        {1,0,0,0},
        {1,3,5,4}
    };

    int max[5][4] = {
        {0,6,5,6},
        {0,6,5,2},
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6}
    };

    int available[4] = {1,6,2,0};

    int need[5][4];
    int finish[5] = {0};
    int safeSeq[5];
    int work[4];

    for(i=0;i<resource;i++)
        work[i] = available[i];

    printf("Need Matrix:\n");

    for(i=0;i<process;i++){
        for(j=0;j<resource;j++){
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ",need[i][j]);
        }
        printf("\n");
    }

    int count = 0;

    while(count < process){
        int found = 0;

        for(i=0;i<process;i++){
            if(finish[i]==0){
```

```
                printf("Need Matrix:\n");

                for(i=0;i<process;i++){
                    for(j=0;j<resource;j++){
                        need[i][j] = max[i][j] - allocation[i][j];
                        printf("%d ",need[i][j]);
                    }
                    printf("\n");
                }

                int count = 0;

                while(count < process){
                    int found = 0;

                    for(i=0;i<process;i++){
                        if(finish[i]==0){
                            int flag = 0;

                            for(j=0;j<resource;j++){
                                if(need[i][j] > work[j]){
                                    flag = 1;
                                    break;
                                }
                            }

                            if(flag==0){
                                for(k=0;k<resource;k++){
                                    work[k] += allocation[i][k];

                                    safeSeq[count++] = i;
                                    finish[i] = 1;
                                    found = 1;
                                }
                            }

                            if(found==0){
                                printf("\nSystem is NOT SAFE\n");
                                return 0;
                            }
                        }
                    }

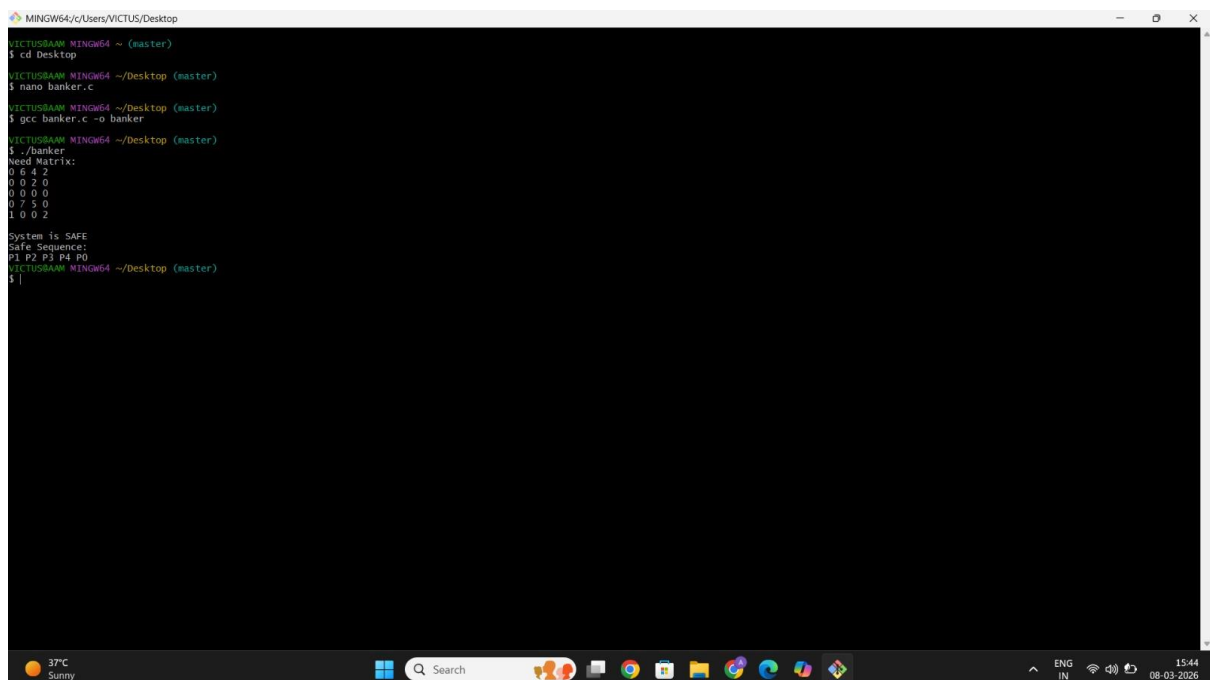
                    printf("\nSystem is SAFE\nSafe Sequence:\n");

                    for(i=0;i<process;i++)
                        printf("%d ",safeSeq[i]);

                    return 0;
                }
            }
        }
    }
}
```

OUTPUT :

1. The program was compiled using the **GCC compiler** with the command `gcc banker.c -o banker`, which created an executable file named **banker**.
2. The compiled program was executed using the command `./banker` in the Git Bash terminal.
3. The output displayed the **Need Matrix**, verified that the system is in a **safe state**, and showed the **safe sequence of processes**.



```
MINGW64/c/Users/VICTUS/Desktop
VICTUS@AM MINGW64 ~ (master)
$ cd Desktop
VICTUS@AM MINGW64 ~/Desktop (master)
$ nano banker.c
VICTUS@AM MINGW64 ~/Desktop (master)
$ gcc banker.c -o banker
VICTUS@AM MINGW64 ~/Desktop (master)
$ ./banker
Need Matrix:
0 6 4 2
0 0 2 0
0 0 0 0
0 7 5 0
1 0 0 2

System is SAFE
Safe Sequence:
P1 P2 P3 P4 P0
VICTUS@AM MINGW64 ~/Desktop (master)
$
```